

The Big 5

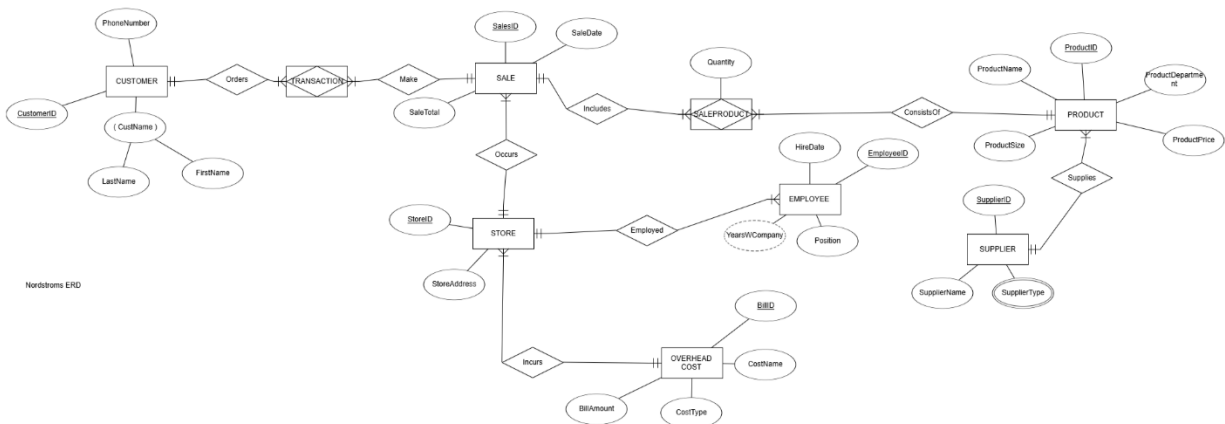
Kayla Goudy, Isaac Joyner, Isaac Morris, Nicholas Mwenye, Sarah Reid

Nordstrom

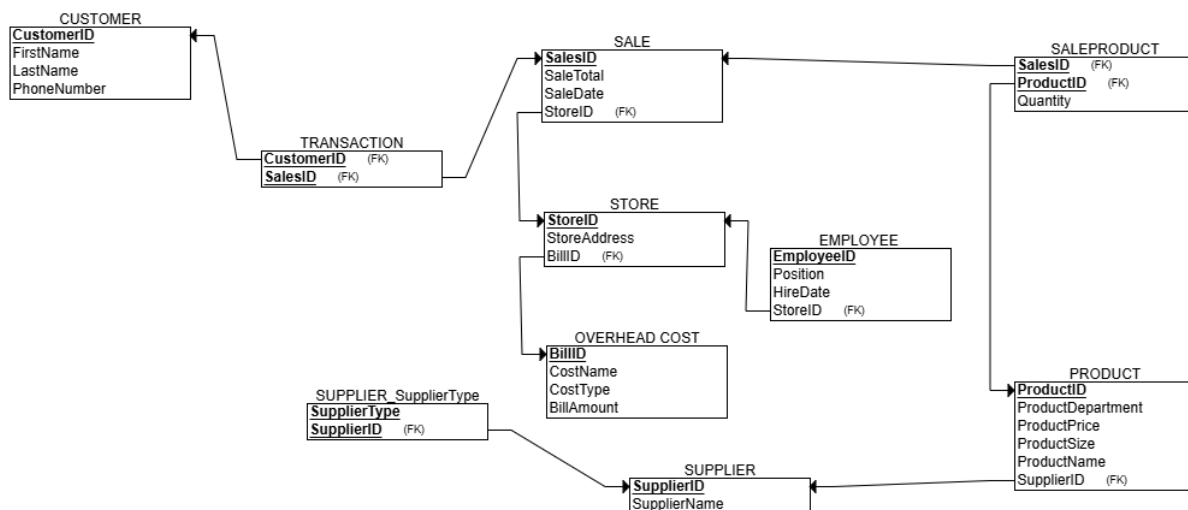
Part I: Business Description

Nordstrom Rack is a leading discount retail store with nearly 300 locations nationwide. It offers brand name designer clothing, shoes, and accessories. It even offers different home goods, such as furniture and kitchenware. The store is aimed towards individuals who are looking for quality products at a reduced cost.

Part II: ER Diagram



Part III: Relational Data Model



Part IV: Data Creation SQL and Data Dictionary

USE bigfive;

DROP TABLE IF EXISTS employee_;

DROP TABLE IF EXISTS transaction;

DROP TABLE IF EXISTS supplier_suppliertype;

DROP TABLE IF EXISTS saleproduct;

DROP TABLE IF EXISTS sale;

DROP TABLE IF EXISTS store;

DROP TABLE IF EXISTS product;

DROP TABLE IF EXISTS overhead_cost;

DROP TABLE IF EXISTS customer;

DROP TABLE IF EXISTS supplier;

CREATE TABLE CUSTOMER

(FirstName CHAR(25) NOT NULL,

LastName VARCHAR(25) NOT NULL,

PhoneNumber VARCHAR(25) NOT NULL,

CustomerID INT NOT NULL,

PRIMARY KEY (CustomerID));

CREATE TABLE OVERHEAD_COST

(CostName varchar(25) NOT NULL,

CostType varchar(25) NOT NULL,

BillAmount INT NOT NULL,

BillID INT NOT NULL,

PRIMARY KEY (BillID));

CREATE TABLE SUPPLIER

(SupplierID INT NOT NULL,

SupplierName VARCHAR(25) NOT NULL,

PRIMARY KEY (SupplierID));

CREATE TABLE PRODUCT

(ProductID INT NOT NULL,

ProductDepartment VARCHAR(25) NOT NULL,

ProductPrice INT NOT NULL,

ProductSize VARCHAR(10) NOT NULL,

ProductName VARCHAR(25) NOT NULL,

SupplierID INT NOT NULL,

PRIMARY KEY (ProductID),

FOREIGN KEY (SupplierID) REFERENCES SUPPLIER(SupplierID));

CREATE TABLE STORE

(StoreID INT NOT NULL,

StoreAddress VARCHAR(30) NOT NULL,

BillID INT NOT NULL,

PRIMARY KEY (StoreID),

FOREIGN KEY (BillID) REFERENCES OVERHEAD_COST(BillID));

CREATE TABLE SUPPLIER_SupplierType

(SupplierType CHAR(25) NOT NULL,

SupplierID INT NOT NULL,
PRIMARY KEY (SupplierType, SupplierID),
FOREIGN KEY (SupplierID) REFERENCES SUPPLIER(SupplierID));

CREATE TABLE EMPLOYEE_
(Position VARCHAR(25) NOT NULL,
EmployeeID INT NOT NULL,
HireDate VARCHAR(25) NOT NULL,
StoreID INT NOT NULL,
PRIMARY KEY (EmployeeID),
FOREIGN KEY (StoreID) REFERENCES STORE(StoreID));

CREATE TABLE SALE
(SalesID INT NOT NULL,
SaleTotal VARCHAR(10) NOT NULL,
SaleDate VARCHAR(25) NOT NULL,
StoreID INT NOT NULL,
PRIMARY KEY (SalesID),
FOREIGN KEY (StoreID) REFERENCES STORE(StoreID));

CREATE TABLE SALEPRODUCT
(Quantity INT NOT NULL,
SalesID INT NOT NULL,
ProductID INT NOT NULL,
PRIMARY KEY (SalesID, ProductID),
FOREIGN KEY (SalesID) REFERENCES SALE(SalesID),

```
FOREIGN KEY (ProductID) REFERENCES PRODUCT(ProductID));
```

```
CREATE TABLE TRANSACTION
```

```
(CustomerID INT NOT NULL,
```

```
SalesID INT NOT NULL,
```

```
PRIMARY KEY (CustomerID, SalesID),
```

```
FOREIGN KEY (CustomerID) REFERENCES CUSTOMER(CustomerID),
```

```
FOREIGN KEY (SalesID) REFERENCES SALE(SalesID));
```

```
SELECT * FROM customer;
```

```
INSERT INTO CUSTOMER VALUES
```

```
('Alice', 'Thompson', '555-1234', 1),
```

```
('Brian', 'Wong', '555-5678', 2),
```

```
('Carla', 'Lopez', '555-7890', 3),
```

```
('David', 'Chen', '555-2468', 4),
```

```
('Eva', 'Williams', '555-1357', 5),
```

```
('Frank', 'Patel', '555-9876', 6),
```

```
('Grace', 'Kim', '555-1122', 7),
```

```
('Hank', 'Nguyen', '555-3344', 8),
```

```
('Ivy', 'Johnson', '555-5566', 9),
```

```
('Jack', 'Brown', '555-7788', 10);
```

```
-- OVERHEAD_COST
```

```
select * from overhead_cost;
```

INSERT INTO OVERHEAD_COST VALUES

('Electricity', 'Utility', 1200.00, 101),
('Water', 'Utility', 350.00, 102),
('Internet', 'Service', 300.00, 103),
('Maintenance', 'Service', 800.00, 104),
('Security', 'Service', 600.00, 105),
('Trash', 'Utility', 200.00, 106),
('Cleaning', 'Service', 450.00, 107),
('Heating', 'Utility', 1000.00, 108),
('A/C', 'Utility', 950.00, 109),
('Insurance', 'Expense', 1200.00, 110);

-- SUPPLIER

INSERT INTO SUPPLIER VALUES

(201, 'Levi Strauss'),
(202, 'Nike Inc.'),
(203, 'Adidas'),
(204, 'Calvin Klein'),
(205, 'Tommy Hilfiger'),
(206, 'Puma'),
(207, 'Steve Madden'),
(208, 'Michael Kors'),
(209, 'Under Armour'),
(210, 'The North Face');

-- PRODUCT

INSERT INTO PRODUCT VALUES

(301, 'Apparel', 59.99, 'M', 'Levi Jeans', 201),
(302, 'Footwear', 89.99, '10', 'Nike Air Max', 202),
(303, 'Apparel', 49.99, 'L', 'Adidas Hoodie', 203),
(304, 'Apparel', 39.99, 'S', 'CK Shirt', 204),
(305, 'Apparel', 69.99, 'M', 'Tommy Polo', 205),
(306, 'Footwear', 79.99, '9', 'Puma Sneakers', 206),
(307, 'Footwear', 99.99, '8', 'Steve Loafers', 207),
(308, 'Accessories', 120.00, 'One', 'MK Bag', 208),
(309, 'Athleisure', 65.00, 'M', 'UA Shorts', 209),
(310, 'Outerwear', 150.00, 'L', 'TNF Jacket', 210);

-- STORE

INSERT INTO STORE VALUES

(401, '123 Fashion Ave, NY', 101),
(402, '456 Style Blvd, LA', 102),
(403, '789 Chic St, Miami', 103),
(404, '321 Luxe Ln, Chicago', 104),
(405, '654 Wardrobe Way, Dallas', 105),
(406, '987 Trendy Rd, Seattle', 106),
(407, '111 Urban Dr, Denver', 107),
(408, '222 Classic Ct, Atlanta', 108),
(409, '333 Elite Sq, Boston', 109),
(410, '444 Mod Ln, Houston', 110);

-- SUPPLIER_SupplierType

```
INSERT INTO SUPPLIER_SupplierType VALUES
```

```
('Clothing', 201),
```

```
('Shoes', 202),
```

```
('Clothing', 203),
```

```
('Clothing', 204),
```

```
('Clothing', 205),
```

```
('Shoes', 206),
```

```
('Shoes', 207),
```

```
('Accessories', 208),
```

```
('Athletic', 209),
```

```
('Outerwear', 210);
```

```
-- EMPLOYEE_
```

```
INSERT INTO EMPLOYEE_ VALUES
```

```
('Manager', 501, '2023-02-10', 401),
```

```
('Cashier', 502, '2024-06-01', 402),
```

```
('Stock Associate', 503, '2024-08-15', 403),
```

```
('Sales Associate', 504, '2023-05-20', 404),
```

```
('Cashier', 505, '2022-12-05', 405),
```

```
('Assistant Manager', 506, '2024-01-30', 406),
```

```
('Sales Lead', 507, '2023-09-10', 407),
```

```
('Fitting Room', 508, '2024-11-12', 408),
```

```
('Customer Service', 509, '2022-10-01', 409),
```

```
('Janitor', 510, '2021-06-18', 410);
```

```
-- SALE
```

INSERT INTO SALE VALUES

(601, 149.98, '2025-04-20', 401),
(602, 59.99, '2025-04-21', 402),
(603, 89.99, '2025-04-22', 403),
(604, 159.98, '2025-04-22', 404),
(605, 39.99, '2025-04-23', 405),
(606, 79.99, '2025-04-23', 406),
(607, 120.00, '2025-04-23', 407),
(608, 150.00, '2025-04-24', 408),
(609, 65.00, '2025-04-24', 409),
(610, 109.98, '2025-04-24', 410);

-- SALEPRODUCT

INSERT INTO SALEPRODUCT VALUES

(1, 601, 301),
(1, 601, 302),
(1, 602, 301),
(1, 603, 302),
(2, 604, 304),
(1, 605, 304),
(1, 606, 306),
(1, 607, 308),
(1, 608, 310),
(2, 610, 303);

-- TRANSACTION

INSERT INTO TRANSACTION VALUES

$(1, 601),$ $(2, 602),$ $(3, 603),$

(4, 604),

 $(5, 605),$ $(6, 606),$ $(7, 607),$ $(8, 608),$

(9, 609),

(10, 610);

Part V: Query Matrix and SQL Queries

[illegible]

-- Question 1

-- List all customers whose name starts with an "I"

```
SELECT * FROM customer
WHERE FirstName like 'I%';
```

-- Question 2

-- List customer name, phone number, and customer id

```
SELECT Customerid, FirstName, LastName, PhoneNumber,
COUNT(customerid) AS customer_count
FROM customer
GROUP BY Customerid, FirstName, PhoneNumber;
```

-- Question 3

-- Employee start date starting "2022 – 11 – 08"

```
SELECT *
FROM EMPLOYEE_
WHERE HireDate >= '2022-11-08';
```

-- Question 4

-- List supplier id, product id and product department and the supplier name.

```
SELECT
    PRODUCT.SupplierID, PRODUCT.ProductID, PRODUCT.ProductDepartment,
    SUPPLIER.SupplierName
FROM PRODUCT
JOIN SUPPLIER ON PRODUCT.SupplierID = SUPPLIER.SupplierID;
```

-- Question 5

-- List of customers from 987 Trendy Rd, Seattle

```
SELECT DISTINCT c.FirstName, c.LastName, c.CustomerID
FROM CUSTOMER c
JOIN TRANSACTION t ON c.CustomerID = t.CustomerID
JOIN SALE s ON t.SalesID = s.SalesID
JOIN STORE st ON s.StoreID = st.StoreID
WHERE st.StoreAddress = '987 Trendy Rd, Seattle';
```

-- Question 6

-- List product name by supplier id

```
SELECT SupplierID, ProductName
FROM PRODUCT
ORDER BY SupplierID;
```

-- Question 7

-- What is the average cost of apparel?

```
SELECT AVG(ProductPrice) AS Average_Apparel_Cost
FROM product
WHERE ProductDepartment = 'Apparel';
```

-- Question 8

-- Group department with average price over \$60

```
SELECT ProductDepartment, AVG(ProductPrice) AS AvgPrice
```

```
FROM PRODUCT
GROUP BY ProductDepartment
HAVING AVG(ProductPrice) > 60;
```

-- Question 9

-- Find stores missing overhead cost

```
SELECT s.*
FROM STORE s
LEFT JOIN OVERHEAD_COST o ON s.BillID = o.BillID
WHERE o.BillID IS NULL;
```

-- Question 10

-- List the names of all suppliers who have not supplied any products that were sold in any sale.

```
SELECT SupplierName
FROM supplier
WHERE supplierID NOT IN (
    SELECT DISTINCT SupplierID
    FROM product
    WHERE ProductID IN (
        SELECT DISTINCT ProductID
        FROM saleproduct) );
```